

INSTITUTO TECNOLÓGICO AUTÓNOMO DE
MÉXICO



Proyecto IDO

MODELADO CON PROGRAMACIÓN LINEAL Y DISEÑO DE
METAHEURÍSTICAS

IÑANKI LIENDO
PATRICIO BARBER
SANTIAGO PRADO
ORLANDO ALMAZÁN

2025

Proyecto IDO

31 de enero de 2026

Índice

1. Problema de Modelado 1	2
1.1. Modelado del problema (fase 1)	2
1.2. Solución del problema (fase 1)	5
1.3. Modelado del problema (fase 2)	6
1.4. Solución del problema (fase 2)	8
1.5. Cambios en la política de reemplazo	8
2. Problema de Modelado 2	8
2.1. Formulación del problema	9
2.2. Solución del problema	11
2.3. Solución del problema empezando con el color NG	12
2.4. Reporte a Gerente	12
3. Programación de Metaheurísticas	13
3.1. Metaheurística 1: Ant Colony Optimization	13
3.1.1. Pseudocódigo	15
3.1.2. Solución Mac Bella	16
3.2. Metaheurística 2: Inserción con permutaciones	16
3.2.1. Pseudocódigo	17
3.3. Metaheurística 3: Recocido simulado	18
3.3.1. Pseudocódigo	19
3.3.2. Solución del problema de MacBella	19
3.4. Resultados en Instancias	20
3.5. Conclusiones	21

1. Problema de Modelado 1

El problema presentado se modeló como un problema de programación lineal. El problema puede dividirse en dos fases: la primera es cuando los costos del tractor solamente se incrementan un 15 por ciento anual, y la segunda fase es cuando los costos del tractor, además del 15 por ciento anual, depende de la edad del tractor. Cabe recalcar que no se formuló como un Problema de la ruta más corta.

1.1. Modelado del problema (fase 1)

La formulación para la fase 1 del problema se presenta a continuación.

Variables

Se tienen las siguientes variables de decisión:

$$x_t = \begin{cases} 1 & \text{si se vende el tractor en el año } t \\ 0 & \text{en otro caso} \end{cases} \quad \forall t \in \{1, 2, \dots, 17\}$$

Además, se definen las siguientes variables y constantes auxiliares que ayudarán a modelar el problema:

Variables	Descripción
v_t	Variable real que representa el valor del tractor en el año $t \in \{1, \dots, 17\}$.
p_t	Variable real que representa el ingreso por la venta del tractor en el año $t \in \{1, \dots, 17\}$.
c_t	Constante que representa el costo de mercado del tractor en el año $t \in \{1, \dots, 17\}$.
m_t	Constante que representa el costo de mantenimiento del tractor en el año $t \in \{1, \dots, 17\}$.
M_t	Constante suficientemente grande utilizada para modelar condiciones lógicas en el año $t \in \{1, \dots, 17\}$.

Cuadro 1: Variables de decisión (fase 1)

Como los costos del tractor en el mercado aumentan en un 5 % anual, se tiene que:

$$\begin{aligned}c_1 &= 43000 \\c_t &= (1 + 0,05)c_{t-1}.\end{aligned}$$

En esta primera fase, los costos de mantenimiento crecen de forma constante sin tomar en cuenta la antigüedad del tractor, con lo cual se tiene:

$$\begin{aligned}m_1 &= 1300, \\m_t &= (1 + 0,15)m_{t-1}.\end{aligned}$$

Las condiciones intuitivas del modelo son:

$$v_t = \begin{cases} (1 - 0,1)c_t & x_t = 1, \\ (1 - 0,07)p_{t-1} & x_t = 0, \end{cases}$$

donde el valor inicial v_1 se calcula considerando que el tractor actual tiene dos años de antigüedad:

$$v_1 = \underbrace{\frac{43000}{(1 + 0,05)^2}}_{\text{valor presente en } t=-2} \cdot (1 - 0,1)(1 - 0,07) \approx 32365.$$

Luego,

$$p_t = \begin{cases} v_t & x_t = 1, \\ 0 & x_t = 0. \end{cases}$$

Formulación matemática del problema

$$\begin{aligned}
\min. \quad & \sum_{t=1}^{17} c_t x_t + m_t - s_t \\
\text{s.a.} \quad & x_t + x_{t+1} \leq 1, \forall t = 1, \dots, 16 \\
& x_t + x_{t+1} + \dots + x_{t+4} \geq 1, \forall t = 1, \dots, 12 \\
& v_t \geq (1 - 0,07)p_{t-1} - M_t x_t, \forall t = 2, \dots, 17 \\
& v_t \leq (1 - 0,07)p_{t-1} + M_t x_t, \forall t = 2, \dots, 17 \\
& v_t \geq (1 - 0,1)c_t - M_t(1 - x_t), \forall t = 1, \dots, 17 \\
& v_t \leq (1 - 0,1)c_t + M_t(1 - x_t), \forall t = 1, \dots, 17 \\
& p_t \leq M_t x_t, \forall t = 1, \dots, 17 \\
& p_t \geq v_t - M_t(1 - x_t), \forall t = 1, \dots, 17 \\
& p_t \leq v_t + M_t(1 - x_t), \forall t = 1, \dots, 17 \\
& x_{17} = 1, \\
& p_1 = 32385, \\
& x_t \in \{0, 1\}, \forall t = 1, \dots, 17 \\
& p_t, v_t \in \mathbb{R}_{\geq 0}, \forall t = 1, \dots, 17
\end{aligned}$$

La función objetivo a minimizar es la función de costos operativos, considerando que hay una ganancia por la decisión de vender el tractor. Como $p_t \leq v_t \leq (1 - 0,1)c_t$ para todo t , una buena elección para M_t es ese valor.

1.2. Solución del problema (fase 1)

A partir del modelo, se encuentra que el tractor debe venderse en los años $t \in \{2, 7, 12, 17\}$, con un costo total aproximado de \$196,213. Los valores se muestran en la siguiente tabla, en miles de pesos:

t	c_t	v_t	m_t
1	43.0	32.6449	1.3
2	45.15	40.6350	1.495
3	47.4075	37.7905	1.7192
4	49.7779	35.1452	1.9771
5	52.2668	32.6850	2.2737
6	54.8802	30.3971	2.6148
7	57.6241	51.8617	3.0068
8	60.5053	48.2318	3.4580
9	63.5306	44.8552	3.9767
10	66.7071	41.7153	4.5732
11	70.0425	38.7952	5.2592
12	73.5446	66.1901	6.0481
13	77.2218	61.5568	6.9553
14	81.0829	57.2478	7.9986
15	85.1308	53.2405	9.1984
16	89.3939	49.5137	10.5782
17	93.8631	84.4772	12.1649

Cuadro 2: Valores de mercado, mantenimiento y venta del tractor

1.3. Modelado del problema (fase 2)

Ahora bien, se requieren variables adicionales para calcular los costos de mantenimiento dependiendo de la antigüedad del tractor, las cuales se muestran en la siguiente tabla:

Variable extra	Descripción
e_t	Variable entera que determina la antigüedad del tractor en el año $t \in \{1, \dots, 17\}$.
n_t	Variable real que calcula el costo de mantenimiento dependiendo de la antigüedad del tractor en el año $t \in \{1, \dots, 17\}$.
z_t	Variable binaria auxiliar que determina si $e_t \leq 1$ o $e_t \geq 2$ para cada año $t \in \{1, \dots, 17\}$.
M'_t	Constante real lo suficientemente grande para cada año $t \in \{1, \dots, 17\}$.
M''	Constante real lo suficientemente grande.

Intuitivamente, las variables de decisión deben cumplir con las siguientes reglas:

$$e_t = \begin{cases} 0 & x_t = 1, \\ e_{t-1} + 1 & x_t = 0. \end{cases}$$

Luego,

$$n_t = \begin{cases} 0,01c_t & e_t = 0, \\ 0,01c_{t-1} & e_t = 1, \\ (1 + 0,03)m_{t-1} & e_t \geq 2, \end{cases}$$

y también,

$$z_t = \begin{cases} 0 & e_t \leq 1, \\ 1 & e_t \geq 2. \end{cases}$$

Así pues, al modelo presentado anteriormente le añadimos las siguientes restricciones:

$$\begin{aligned}
e_t &\geq e_{t-1} + 1 - M''x_t, \forall t = 2, \dots, 17 \\
e_t &\leq e_{t-1} + 1 + M''x_t, \forall t = 2, \dots, 17 \\
e_t &\geq 2 - M''(1 - z_t), \forall t = 1, \dots, 17 \\
e_t &\leq 1 + M''z_t, \forall t = 1, \dots, 17 \\
n_t &\geq 0,01c_t(1 - e_t) + 0,01c_{t-1}e_t - M'_tz_t, \forall t = 2, \dots, 17 \\
n_t &\leq 0,01c_t(1 - e_t) + 0,01c_{t-1}e_t + M'_tz_t, \forall t = 2, \dots, 17 \\
n_t &\geq (1 + 0,03)m_{t-1} + M'_t(1 - z_t), \forall t = 2, \dots, 17 \\
n_t &\leq (1 + 0,03)m_{t-1} - M'_t(1 - z_t), \forall t = 2, \dots, 17 \\
e_t &\in \mathbb{Z}_{\geq 0}, \forall t = 1, \dots, 17 \\
z_t &\in \{0, 1\}, \forall t = 1, \dots, 17 \\
n_t &\in \mathbb{R}_{\geq 0}, \forall t = 1, \dots, 17
\end{aligned}$$

No es difícil ver que $M'_t = 4c_t$ y $M'' = 5$ son las mejores cotas superiores.

1.4. Solución del problema (fase 2)

Una vez calculados los resultados, encontramos que la decisión no cambia y por lo tanto los años de venta son los mismos, así como los costos de mercado, de mantenimiento inicial y los valores del tractor. A continuación se presenta solamente los costos adicionales incurridos. De igual forma se presentan en miles de pesos. El costo total incurrido por la decisión óptima es \$122,084.

t	n_t	t	n_t
1	1.3000	10	4.5732
2	1.4950	11	5.2592
3	1.7192	12	6.0481
4	1.9771	13	6.9553
5	2.2737	14	7.9986
6	2.6148	15	9.1984
7	3.0070	16	10.5782
8	3.4580	17	12.1649
9	3.9767		

1.5. Cambios en la política de reemplazo

Las políticas de reemplazo, en efecto, cambian. Como los costos totales son crecientes y dado que no va a haber una última ganancia de venta, se sigue que la mejor estrategia es vender el tractor inmediatamente y comprar uno nuevo cada cinco años. Es decir, la estrategia óptima es agotar la vida útil del tractor de acuerdo a la política de no poder usarlo por más de este plazo.

2. Problema de Modelado 2

Cosméticos Mac Bella cuenta con una línea de producción especializada en la fabricación de pintalabios en 10 diferentes tonos, que van desde el Rosa Pálido hasta el Negro Gótico. Cada semana, la planeación de la producción

se realiza con base en los pedidos programados para la semana siguiente. Sin embargo, el jefe de la planta ha notado que, aun cuando los pedidos son idénticos, los tiempos de entrega varían significativamente dependiendo del orden en que se organiza la producción.

Esta variabilidad se debe a que el tiempo requerido para preparar la línea de producción entre un tono y otro no es constante. En particular, el tiempo de limpieza y ajuste al cambiar de un color a otro depende de qué colores estén involucrados. Por ejemplo, el cambio entre el Rosa Pálido y el Rosa Pop requiere mucho menos tiempo que el cambio entre el Rosa Pálido y el Rojo Ruso o el Negro Gótico.

La empresa proporciona la siguiente tabla, que contiene los tiempos promedio (en horas) necesarios para preparar la línea al cambiar de un color a otro:

Pinta Labios	Código	R1	R2	R6	RW	RR	SS	SP2	S12	T3	NG	T.P.*
Rosa Pálido	R1	0	2	2	3	4	5	4	3	5	6	60
Rosa Pop	R2	2	0	1	3	3	4	6	4	3	5	31
Rosa Discreto	R6	2	1	0	4	5	3	2	4	1	7	45
Ruby Woo	RW	3	3	4	0	2	4	5	3	5	2	60
Rojo Ruso	RR	4	3	5	2	0	2	4	1	2	2	71
Saigon Summer	SS	5	4	3	4	2	0	3	2	1	3	82
Sheer Plum	SP2	4	6	2	5	4	3	0	3	2	1	52
Snob	S12	3	4	4	3	1	2	3	0	5	3	44
Tanarama	T3	5	3	1	5	2	1	2	5	0	4	74
Negro Gótico	NG	6	5	7	2	2	3	1	3	4	0	100

Figura 1: Tiempos promedio de preparación entre tonos de pintalabios

Con base en esta información, se busca una solución eficiente del orden de producción de los pintalabios. Así pues, el objetivo de este reporte es determinar tal orden formulando el problema como un Problema de programación lineal y modelarlo como un Problema del agente viajero.

2.1. Formulación del problema

Con los datos dados, se tiene el siguiente problema de programación lineal:

Variables de decisión:

$$x_{ij} = \begin{cases} 1 & \text{si el color } j \text{ se produce inmediatamente después del color } i \\ 0 & \text{en otro caso} \end{cases} \quad \forall i, j \in \{1, 2, \dots, 10\}$$

Función objetivo: Minimizar el tiempo total de preparación entre cambios de color:

$$\text{mín} \sum_{i=1}^{10} \sum_{\substack{j=1 \\ j \neq i}}^{10} c_{ij} \cdot x_{ij}$$

Restricciones: Primero deben agregarse las siguientes restricciones, las cuales establecen que después del color i solo se produce un color y que antes del color j se produce solo un color:

$$\sum_{\substack{j=1 \\ j \neq i}}^{10} x_{ij} = 1 \quad \forall i = 1, \dots, 10$$

$$\sum_{\substack{i=1 \\ i \neq j}}^{10} x_{ij} = 1 \quad \forall j = 1, \dots, 10$$

La siguiente restricción establece que deben producirse todos los 10 colores (sin repeticiones):

$$\sum_{i \in S} \sum_{j \in S, j \neq i} x_{ij} \leq |S| - 1 \quad \forall S \subset \{1, 2, \dots, 10\}, 2 \leq |S| \leq 9$$

Naturaleza de las variables: Las variables de decisión determinan si el color j se produce inmediatamente después del color i , por lo cual su naturaleza es binaria, es decir:

$$x_{ij} \in \{0, 1\} \quad \forall i, j = 1, \dots, 10, i \neq j$$

Analizando el problema, es relativamente sencillo notar que es equivalente al Problema del agente viajero (TSP). Se tienen las siguientes equivalencias:

- Cada uno de los 10 colores es un nodo del TSP (una ciudad)
- El tiempo de producción entre el color i y el color j es el costo de la arista entre el nodo i y el nodo j del TSP (la distancia entre las ciudades i y j)

- Se busca producir todos los colores una vez, es decir, pasar por todos los nodos solo una vez y regresar al original (en el TSP se busca recorrer todas las ciudades una vez y regresar a la ciudad de origen)
- Se desea minimizar el tiempo de producción, es decir, buscar la solución que minimice los costos de las aristas entre nodos (en el TSP se desea minimizar las distancias)
- Las primeras dos restricciones establecen que después del labial i debe producirse solo un labial j, lo cual en el TSP es que desde la ciudad i se visita solamente la ciudad j
- La última restricción establece que deben producirse todos los pintalabios regresando al primer pintalabios que se produjo, esto en el TSP es equivalente a recorrer todas las ciudades una sola vez y regresar a la primera (esta condición es para eliminar subtours)

2.2. Solución del problema

A partir de la formulación anterior (usando el hecho de que el problema es análogo al Problema del agente viajero) y utilizando Julia, se llegó a la solución que se muestra en el siguiente diagrama.

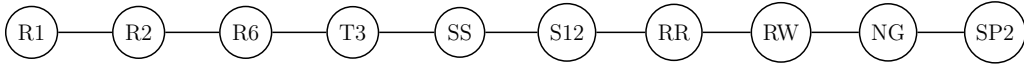


Figura 2: Diagrama de transición entre colores (semana 1)

La solución anterior podría implementarse para la primera semana, posteriormente para la siguiente semana solamente se invierte el orden, es decir, como se muestra en el siguiente diagrama.

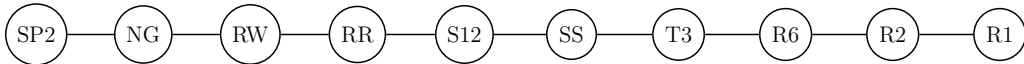


Figura 3: Orden de producción de colores de labiales

Así pues, puede usarse el procedimiento anterior: comenzar la primera semana produciendo el color R1 (rosa pálido), luego el color R2 (rosa pop) y así hasta llegar al color SP2 (sheer plum). Posteriormente, en la segunda

semana, empezar con SP2 (sheer plum), luego NG (negro gótico) y así hasta llegar a R1 (rosa pálido). Posteriormente, en la semana tres puede repetirse el modelo de producción de la semana 1, en la semana 4 el de la semana 2 y así sucesivamente. Para esta solución óptima, el valor de la función objetivo es 13, es decir, es el tiempo que se tarda la producción.

2.3. Solución del problema empezando con el color NG

En el escenario de que tenga que empezarse la producción con el NG (negro gótico) se tiene la siguiente solución:

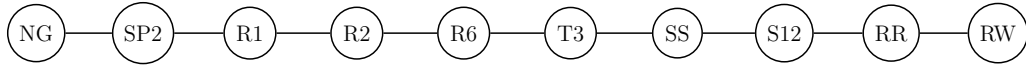


Figura 4: Diagrama de transición entre colores empezando con NG

Al igual que el el inciso b), la siguiente semana simplemente se invierte el orden, empezando ahora con RW, es decir:

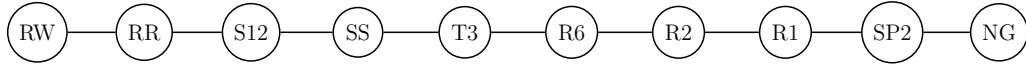


Figura 5: Diagrama de transición de colores (de RW a NG)

En este contexto, con la solución óptima, el valor de la función objetivo es 15, que es el tiempo de producción.

2.4. Reporte a Gerente

El problema de producción de la gama de colores se puede ver como un problema del Agente Viajero. Este famoso problema matemático consiste en encontrar la ruta más corta entre una colección de ciudades. En nuestro caso, los diferentes colores representan nuestras ciudades, y el tiempo de preparación maquinaria entre cualesquiera dos colores representa la distancia entre estas dos ciudades.

Esta formulación del problema tiene varias ventajas. Primero que nada, su formulación como un problema de programación lineal es relativamente fácil, y existen restricciones auxiliares que nos garantizan una solución óptima.

La formulación también nos permite modificar las variables del modelo para situaciones futuras. Si se desea agregar un nuevo labial, solamente es necesario indicar los tiempos que se tarda la limpieza dependiendo del siguiente labial que se va a producir. Por ejemplo, si el gerente busca agregar un labial Toxic Green, necesitará ingresar en la matriz de distancias los tiempos que se tarda la maquinaria para que el siguiente labial sea o bien Negro Gótico, o bien Red Plum, etcétera. Innovaciones tecnológicas que reduzcan las ventanas de tiempo entre la producción solo representan un cambio en los tiempos que damos de entrada al modelo, y en este caso sería actualizar la matriz de costos. Nuestra formulación en programación lineal es adaptable a un número arbitrario de labiales. El modelo resuelve fácilmente el problema de optimización para hasta 1,000 colores de labial.

Ahora bien, un problema que se podría presentar es el costo computacional y el tiempo requerido por el algoritmo. Afortunadamente, existe mucha literatura que propone diferentes soluciones más artesanales que pueden ser menos costosas, las cuales llamamos *metaheurísticas*. A continuación presentamos de forma más técnica tres de estas metaheurísticas.

3. Programación de Metaheurísticas

3.1. Metaheurística 1: Ant Colony Optimization

Esta metaheurística se basa en la forma en la que las hormigas forman rutas en su ambiente a través de feromonas. Esta metaheurística fue implementada por primera vez por Marco Dorigo en 1992 en su tesis doctoral [3]. La optimización por colonia de hormigas (ACO) consiste en crear agentes (hormigas) que viajan de una ciudad i a una ciudad j basándose en una regla de decisión que refleja la probabilidad de que una hormiga tome un camino dependiendo de la cantidad de feromonas que hay en el camino y en la longitud del camino. Una forma de esta regla puede ser como la usada por Dorigo [3]:

$$P_{ij} = \frac{\tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}}{\sum_k \tau_{ik}^{\alpha} \cdot \eta_{ik}^{\beta}}$$

Donde:

- P_{ij} : la probabilidad de moverse de la ciudad i a la ciudad j .
- τ_{ij} : la cantidad de feromonas en el camino de i a j .
- α : parámetro que controla la influencia de las feromonas en la decisión de la hormiga.
- η_{ik} : atractivo del camino de i a j , que es inversamente proporcional a la distancia entre i y j .
- β : parámetro que controla la influencia de η_{ik} en la decisión de la hormiga.
- La suma del denominador se hace sobre todas las k ciudades no visitadas desde la ciudad actual i .

Además de esta regla de decisión, se debe considerar la forma en la que las feromonas se evaporan del camino (esto inspirado en el mecanismo real de las colonias de hormigas) y también cómo se va reforzando el rastro de feromonas en una ruta cuando muchas hormigas han elegido esa ruta.

Para simular la evaporación de feromonas usamos la fórmula:

$$\tau_{ij} = (1 - \rho) \cdot \tau_{ij}$$

Y para la adición de feromonas en el camino usamos:

$$\tau_{ij} = \tau_{ij} + \Delta\tau_{ij}$$

Donde:

- $\Delta\tau_{ij} = \sum_k \frac{Q}{L_k}$
- ρ : es la tasa de evaporación de las feromonas (va de 0 a 1).
- L_k : longitud del recorrido realizado por la hormiga k .
- La suma se realiza sobre todas las hormigas k que han utilizado el camino de i a j en su recorrido.

- Por simplicidad hacemos $Q = 1$ para no tener que manejar ese parámetro extra. Q indica qué tanto afecta la distancia en la actualización paso a paso del peso de la feromona.

Como se lleva a cabo la optimización es que inicializas el proceso con una cantidad arbitraria de feromonas en todos los caminos(aristas) entre ciudades(nodos). Colocas una hormiga en una ciudad arbitraria y dejas que complete un tour siguiendo la regla de decisión dada. Repites este proceso iniciando en una ciudad distinta cada vez hasta haber pasado por todas. Al final de esta iteración, usas las reglas de feromonas para evaporar rastros de feromonas en caminos menos usados y amplificar las feromonas en los caminos más usados. Repites este procesos con el número de iteraciones que hayas establecido desde un principio. La ruta más corta se va evaluando al pasar cada iteración y al final del proceso te quedas con la ruta más corta hallada.

3.1.1. Pseudocódigo

Algorithm 1 Pseudocódigo del Algoritmo de la Colonia de Hormigas

Require: Vector de coordenadas X,Y

```

while el criterio de parada no se cumpla(no. de iteraciones) do
  Posicionar cada hormiga en un nodo inicial
  repeat
    for cada hormiga do
      Elegir siguiente nodo aplicando la regla de transición de estado
      Guardar actualización de feromonas paso a paso (Adición)
    end for
  until cada hormiga haya construido una solución
  Actualizar la mejor solución
  Aplicar actualización de feromonas de manera global
end while
return Mejor solución encontrada

```

Este pseudocódigo es una adaptación del usado por Kumar y Reddy [1].

3.1.2. Solución Mac Bella

Usando esta metaheurística en el problema del capítulo 2, obtenemos el siguiente orden óptimo de producción:

Rosa Discreto $\rightarrow$ Rosa Pop $\rightarrow$ Rosa Pálido $\rightarrow$ Ruby Woo $\rightarrow$
Negro Gótico $\rightarrow$ Sheer Plum $\rightarrow$ Snob $\rightarrow$ Rojo Ruso $\rightarrow$ Saigon
Summer $\rightarrow$ Tanarama $\rightarrow$ Rosa Discreto

Y la distancia del recorrido es 17.

3.2. Metaheurística 2: Inserción con permutaciones

Usamos una heurística de inserción en la que escogemos como *subtour* inicial el envolvente convexo de las ciudades. Luego, dado un punto en el interior, lo agregamos al *subtour* de tal manera que el orden en que se añade minimiza el costo de agregarlo en cualquier otro orden. Repetimos esto para cada punto en el interior y finalizamos el algoritmo una vez que todos los puntos sean parte del *subtour*, en cuyo caso ya deviene en el *tour* completo. Por la construcción misma del algoritmo tenemos que no se forma ningún *subtour* más allá del inicial.

Como la ruta generada depende del orden en el que añadimos los puntos y, dado que el algoritmo es de orden cuadrático así como altamente paralelizable, corremos el proceso alrededor de 800 veces permutando los puntos interiores. De esas ochocientas soluciones obtenidas, escogemos la que minimice la distancia global. [4]

Cabe mencionar que el método de post-procesamiento 2-opt no hace ningún efecto, pues la forma en que se añaden los puntos desde un inicio evita los cruzamientos de rutas. Las versiones iniciales de este algoritmo hacían uso de 3-opt para disminuir errores locales, en el sentido de que conectaba dos puntos cercanos pero que se añadían en aristas distintas. A pesar de que la distancia total disminuía, el tiempo que se tardaba en finalizar aumentaba significativamente, por lo que decidimos no implementar este post-procesamiento en la versión final.

Para el desarrollo de esta heurística **no** usamos chat-gpt.

3.2.1. Pseudocódigo

Algorithm 2 Metaheurística basada en la envolvente convexa para TSP

```
1: Entrada: Subrecorrido inicial  $S_{\text{init}}$ , coordenadas  $C$ , matriz de distancias  
    $M$ , permutación aleatoria permute  
2: Salida: Ruta generada  $S$   
  
3:  $S \leftarrow S_{\text{init}}$  ▷ Copiar el subrecorrido inicial  
4:  $n \leftarrow |C|$  ▷ Número total de nodos  
5:  $V \leftarrow \{0, 1, \dots, n - 1\} \setminus S$  ▷ Nodos por insertar  
6: if permute then  
7:   Permutar aleatoriamente  $V$   
8: end if  
9: for  $v \in V$  do  
10:    $p \leftarrow \text{InserciónMínima}(S, v, M)$   
11:   Insertar  $v$  en  $S$  en la posición  $p$   
12: end for  
13: return  $S$ 
```

Nota: La adaptación de esta metaheurística al problema de producción de labiales es más complicada, ya que el algoritmo requiere las posiciones geográficas de las ciudades (vectores en $\mathbb{R}^n$).

3.3. Metaheurística 3: Recocido simulado

La metaheurística del Recocido simulado (o Simulated Annealing en inglés) es un algoritmo de búsqueda local que toma inspiración en la metalurgia. Se intenta simular el enfriamiento gradual de un metal, proceso que logra conseguir una configuración molecular de energía mínima (estructura cristalina).

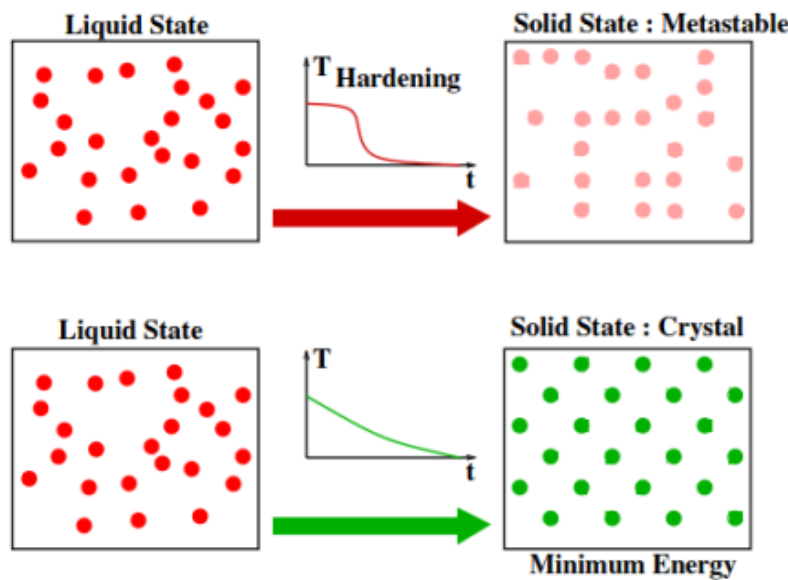


Figura 6: Visualización gráfica del recocido. Al disminuir lentamente la temperatura, se logra alcanzar la configuración óptima. Fuente: Delahaye, Chaimatanan, Mongeau (2009)[2]

El algoritmo encuentra un camino óptimo realizando búsquedas locales a partir de una solución propuesta y una vecindad de soluciones similares. Si encontramos una solución cercana que mejore nuestra función objetivo (en este caso la distancia del camino), nos cambiamos a esta y volvemos a iterar el proceso. Una dificultad que se presenta con estos algoritmos de búsqueda local es la posibilidad de caer en mínimos locales. Si en la vecindad que realizamos una búsqueda la función objetivo es convexa, el algoritmo acaba en un mínimo que no necesariamente es el global. Para evitar esto, el algoritmo de recocido simulado también permite el cambio a una solución peor. Este cambio ocurre con cierta probabilidad, la cual depende de el nivel de tem-

peratura (una variable que disminuimos con cada iteración) y el tamaño del error.

3.3.1. Pseudocódigo

Algorithm 3 Recocido Simulado para TSP (basado en R)

```

1: Entrada: Matriz de distancias  $M$ , temperatura inicial  $T$ , factor de enfriamiento  $\alpha$ , número máximo de iteraciones  $k_{\text{máx}}$ 
2: Salida: Mejor solución encontrada  $S_{\text{mejor}}$ 

3: Generar solución inicial  $S$  (camino aleatorio)
4:  $S_{\text{mejor}} \leftarrow S$ 
5:  $k \leftarrow 0$ 
6: while  $T > 0$  y  $k \leq k_{\text{máx}}$  do
7:   Generar vecino  $S'$  intercambiando dos nodos aleatorios de  $S$ 
8:   Calcular  $\Delta \leftarrow E(S) - E(S')$ 
9:   if  $\Delta \geq 0$  then
10:     $S \leftarrow S'$ 
11:    if  $E(S') < E(S_{\text{mejor}})$  then
12:       $S_{\text{mejor}} \leftarrow S'$ 
13:    end if
14:  else
15:    Generar  $r \sim \mathcal{U}(0, 1)$ 
16:    if  $r < \exp(\Delta/T)$  then
17:       $S \leftarrow S'$ 
18:    end if
19:  end if
20:   $T \leftarrow \alpha \cdot T$ 
21:   $k \leftarrow k + 1$ 
22: end while
23: return  $S_{\text{mejor}}$ 

```

3.3.2. Solución del problema de MacBella

Utilizando R, se programó el código para esta metaheurística y se llegaron a los siguientes resultados.

Con 500001 iteraciones se llega a la siguiente solución, en un tiempo de 5.492743 segundos, y el valor de la función objetivo es 14.

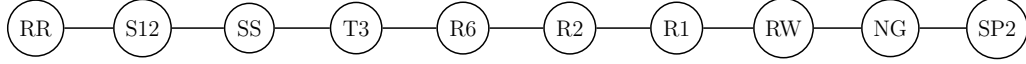


Figura 7: Orden de producción de colores de labiales

Para la segunda parte del problema, cuando se pide que inicie con el color negro se obtuvieron los siguientes resultados. El algoritmo utilizó 50001 iteraciones, tardó 0.5118201 segundos y da un valor de la función objetivo de 15. Se obtuvo la siguiente solución:

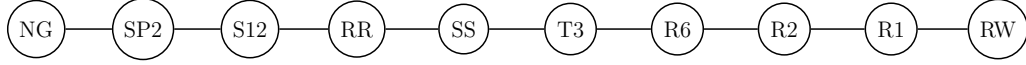


Figura 8: Orden de producción de colores de labiales iniciando en NG

3.4. Resultados en Instancias

Instancia	Ant colony	Inserción	Recocido S	PL Exacto
	Distancia / Tiempo	Distancia / Tiempo	Distancia / Tiempo	Distancia / Tiempo
Inst1(29)	9233 / 4.3m	9074 / 0.2393s	25814 / 1.005s	9074 / 3.387s
Inst2(48)	35425 / 1.21m	33619 / 0.2703s	157530 / 1.101s	33523 / 3751.610s
Inst3(51)	456 / 17.9m	430 / 0.2062s	1313 / 1.259s	428 / 23.619s
Inst4(52)	7664 / 12.8m	7544 / 0.2755s	22205 / 1.356s	7544 / 14.855s
Inst5(70)	711 / 16.3m	682 / 0.4199s	3410 / 1.521s	677 / 2990.96s
Inst6(1979)	NC / NC	93561 / 38.6133s	1080606 / 840s	86888 / 747.835s
MacBella	17 / 0.5m	NA / NA	15 / 0.512s	13 / 0.04s

Cuadro 3: Comparación de metaheurísticas y PL

Cabe mencionar que para la solución del problema lineal para la última instancia (1979) se utilizó Concorde en vez de Julia.

3.5. Conclusiones

El método de la inserción resultó ser el más rápido, en particular, fue diez veces más rápido comparado con el método de recocido simulado, y aproximadamente 60 veces más rápido que la optimización de hormigueros. Asimismo, tuvo mejoras significativas en cuanto a las distancias encontradas por los otros dos métodos. De hecho, tuvo la solución exacta en cuanto a las instancias 4 y 1, mientras que en las otras tuvo un error relativo acotado superiormente por el 7 % (instancia 6).

Lo anterior muestra que en ocasiones es mejor empezar con métodos conceptualmente simples y austeros en implementación. Cabe resaltar que la eficacia del método de la inserción recae principalmente en su alta paralelización. Una posible dirección de mejora es agregar una técnica de post-procesamiento que sea de orden menor o igual a $O(n^2)$, de tal forma que la complejidad del algoritmo siga siendo la misma.

Recordemos que la metaheurística de la inserción solamente funciona cuando los puntos en el plano sí tienen sentido geográfico. De ahí nació la imposibilidad de aplicarlo a problemas más generales tales como el de la producción de labiales "MacBella".

Debido a que tuvimos acceso a las soluciones óptimas gracias a los pequeños tamaños de los problemas, pudimos hacer comparaciones exactas. No obstante, es necesario realizar múltiples análisis para problemas de mayor complejidad (por ejemplo, con mayor número de nodos, o donde los nodos no estén distribuidos uniformemente). En estos casos, resultaría sensato agregar pre-procesamientos, tales como detección de clusters o algoritmos conocidos como "divide-and-conquer". De esta manera, podríamos generar subproblemas y resolverlos con el algoritmo de la inserción. Así pues, hay cabida para una mayor paralelización. Sin embargo, habría que consolidar los *clusters* generados con una generalización del método de la inserción.

Referencias

- [1] Abbas Afshar and Shahrbanou Madadgar. Ant colony optimization for continuous domains: Application to reservoir operation problems. In *2008 Eighth International Conference on Hybrid Intelligent Systems*, pages 13–18, 2008.

- [2] Daniel Delahaye, Supatcha Chaimatanan, and Marcel Mongeau. *Simulated Annealing: From Basics to Applications*, pages 1–35. Springer International Publishing, Cham, 2019.
- [3] Marco Dorigo, Mauro Birattari, and Thomas Stutzle. Ant colony optimization. *IEEE Computational Intelligence Magazine*, 1(4):28–39, 2006.
- [4] Christian Nilsson. Heuristics for the traveling salesman problem. 2003.